



Software Development Assessment 2

Document ID: LUB-AC-003

Common Requirements for ALL Projects:

Your console application MUST demonstrate:

1. Class Structure (Separate Files)

- **Create at least 3-4 classes in separate .cs files**
- **Each class should be in its own file with proper namespace**

2. Access Modifiers (All four types must appear)

- **public** - for methods that need to be accessible from anywhere
- **private** - for fields and helper methods
- **protected** - at least one method that could be used by derived classes
- **internal** - at least one method/property accessible within the same assembly

3. Control Statements

- **if-else** - for decision making
- **switch** - for menu or multi-way branching
- **for loop** - for iterating through arrays
- **while loop** - for the main program loop or repeated operations

4. Data Structures

- **Array** - for fixed-size collections (e.g., movie list, product catalog)
- **ArrayList** - for dynamic collections (e.g., bookings, cart items)

5. Console Application

- **Interactive menu-driven interface**
- **Clear user prompts and feedback messages**
- **Error handling for invalid inputs**

Project-Specific Instructions

Choose ONE of the following projects and implement it according to the instructions below:

Project 1: Movie Ticket Booking System

Core Requirements:

Classes to create:

- **Movie.cs** - Represents a movie (Title, Genre, Duration, Price, AvailableSeats)
- **Booking.cs** - Represents a ticket booking (BookingID, Movie, CustomerName, NumberOfTickets, BookingTime)
- **Theater.cs** - Manages movies and bookings (main logic class)

Features to implement:

?

Feature	Required Concepts
Display all available movies	for loop, Array
Book tickets for a movie	if for seat availability, ArrayList for bookings
View all bookings	for loop, ArrayList
Cancel a booking	if to validate, ArrayList removal
Search movies by genre	if, for loop

Sample Menu Structure:

text

MOVIE TICKET BOOKING SYSTEM

1. View All Movies
2. Book Tickets
3. View My Bookings
4. Cancel Booking

5. Search Movies by Genre

6. Exit

Additional Requirements:

- **Validate seat availability before booking**
- **Prevent booking more seats than available**
- **Generate a unique booking ID for each booking**
- **Show total price calculation**

Project 2: Supermarket POS System

Core Requirements:

Classes to create:

- **Product.cs** - Product details (ID, Name, Price, Quantity)
- **CartItem.cs** - Item in shopping cart (Product, Quantity)
- **Supermarket.cs** - Main system logic
- **Receipt.cs** - Generate receipts

Features to implement:

?

Feature	Required Concepts
Display available products	for loop, Array
Add items to cart	ArrayList for cart
Update item quantity	if validation
Remove from cart	ArrayList operations
Calculate total	for loop, arithmetic
Generate receipt	switch for payment method

Sample Menu Structure:

text

SUPERMARKET POS SYSTEM

1. Browse Products
2. View Cart
3. Add Item to Cart

4. Update Item Quantity

5. Remove Item from Cart

6. Checkout & Generate Receipt

7. Exit

Additional Requirements:

- **Implement at least 5 products in the product array**
- **Apply discount if total exceeds certain amount**
- **Accept different payment methods (Cash/Card) using switch**
- **Display itemized receipt with subtotal, tax, and total**

Project 3: Parking Management System

Core Requirements:

Classes to create:

- **Vehicle.cs** - Vehicle details (LicensePlate, VehicleType, EntryTime)
- **ParkingSlot.cs** - Slot information (SlotNumber, IsOccupied, Vehicle)
- **ParkingLot.cs** - Main management logic
- **Ticket.cs** - Parking ticket details

Features to implement:

?

Feature	Required Concepts
Register vehicle entry	ArrayList for vehicles
Calculate parking fee	if for rate calculation
Process vehicle exit	for to find vehicle
Display parked vehicles	for loop
Check available slots	if condition
Main system loop	while

Sample Menu Structure:

text

PARKING MANAGEMENT SYSTEM

1. Vehicle Entry

2. Vehicle Exit

3. View Parked Vehicles

4. Check Available Slots

5. View Today's Revenue

6. Exit

Additional Requirements:

- **Different rates for different vehicle types (Car, Motorcycle, Truck) using switch**
- **Calculate fee based on hours parked**
- **Maximum capacity limit (use array size)**
- **Display parking duration in hours**

Project 4: Quiz / Exam System

Core Requirements:

Classes to create:

- Question.cs - Question details (Text, Options, CorrectAnswer, Points)
- Quiz.cs - Manages quiz questions
- Participant.cs - Student/taker information
- Result.cs - Score and feedback

Features to implement:

?

Feature	Required Concepts
Display questions	for loop, Array
Capture answers	switch for options
Calculate score	if for checking
Show results	ArrayList for responses
Review answers	for with comparison

Sample Menu Structure:

text

QUIZ EXAM SYSTEM

1. Start New Quiz
2. View All Questions
3. Add Question (Admin)
4. View Past Results

5. Exit

Additional Requirements:

- **Minimum 5 questions in question array**
- **Multiple choice questions with 4 options**
- **Display correct/incorrect feedback after each question (optional)**
- **Show percentage score at the end**
- **Store participant name and score in ArrayList**

Project 5: Hospital Patient Management System

Core Requirements:

Classes to create:

- Patient.cs - Patient details (ID, Name, Age, Gender, Contact)
- Doctor.cs - Doctor information (ID, Name, Specialization)
- Appointment.cs - Appointment details
- Hospital.cs - Main management system

Features to implement:

?

Feature	Required Concepts
Register new patient	ArrayList for patients
Assign doctor to patient	if for availability
Display all patients	for loop
Search patient by name/ID	if comparison
List doctors by specialization	for, Array
Main menu loop	while

Sample Menu Structure:

text

HOSPITAL PATIENT MANAGEMENT SYSTEM

1. Register New Patient

2. Assign Doctor

3. View All Patients

4. Search Patient

5. View Doctors by Specialization

6. Discharge Patient

7. Exit

Additional Requirements:

- **Pre-populate doctor array with at least 3 doctors**
- **Validate patient age (must be >0 and <150)**
- **Generate unique patient ID**
- **Display patient's assigned doctor**

Project 6: Restaurant Ordering System

Core Requirements:

Classes to create:

- MenuItem.cs - Item details (ID, Name, Category, Price)
- Order.cs - Order information (OrderID, Items, Total, Status)
- Table.cs - Table management
- Restaurant.cs - Main system logic

Features to implement:

?

Feature	Required Concepts
Display menu by category	switch, for loop
Place order	ArrayList for orders
Add items to existing order	if validation
Calculate bill	for loop
View all orders	ArrayList
Main program loop	while

Sample Menu Structure:

text

RESTAURANT ORDERING SYSTEM

1. View Menu

2. Place New Order

3. Add Items to Order

4. View Bill

5. View All Orders

6. Complete Order (Checkout)

7. Exit

Additional Requirements:

- **Menu categories: Appetizers, Main Course, Desserts, Beverages**
- **Tax calculation (e.g., 10% service tax)**
- **Table number assignment**
- **Display order summary with itemized list**